

# Technical Interview: Coding Challenge

## Build a Live Poll App

### Overview

As part of our interview process, we'd like you to build a small **live voting/poll application**. The goal is not pixel-perfect design — it's to see how you think about architecture, make technical decisions, and build working software.

**You are welcome (and encouraged) to use AI tools** like ChatGPT, Claude, Copilot, or anything else. We care about the result and your understanding of it, not whether every line was typed by hand. During the interview, we'll walk through the code together and discuss your decisions.

---

### What to Build

A web application where:

1. A user can **create a poll** with a question and 2–5 answer options
2. The poll generates a **shareable link** (or unique ID)
3. Other users can open the link and **cast a vote**
4. The **results** are displayed visually (e.g. progress bars, a simple chart)

That's it. Keep it simple, keep it working.

---

### Requirements

#### Must have:

- A frontend with at least two views: one for voting, one for results
- A backend API (REST or GraphQL — your choice)
- A database storing polls, options, and votes
- The app should be runnable locally with clear setup instructions

#### Nice to have (not required):

- Live-updating results (WebSockets, SSE, or polling)
- Deployed somewhere we can access (Vercel, Railway, Fly.io, etc.)
- Basic input validation
- Some visual polish

### Explicitly not required:

- User authentication or login
  - Comprehensive test coverage
  - Production-grade error handling
  - Mobile responsiveness
- 

## Tech Stack

Use whatever you're most comfortable with. Some common choices:

| Layer      | Examples                                          |
|------------|---------------------------------------------------|
| Frontend   | React, Vue, Svelte, Next.js, plain HTML/JS        |
| Backend    | Node/Express, FastAPI, Django, Go, Rails, Laravel |
| Database   | PostgreSQL, SQLite, MongoDB, even a JSON file     |
| Deployment | Vercel, Railway, Fly.io, Render, Docker           |

There is no "right" stack. We're interested in *why* you chose what you chose.

---

## Time Expectation

This should take roughly up to **2 hours**. Please don't spend a whole weekend on it. A clean, working MVP is much better than an overengineered half-finished project.

---

## What to Prepare for the Interview

During our conversation, we'll go through the code together. Be ready to talk about:

- **Your architecture** — How is the project structured and why?
- **Your data model** — What does your database schema look like?
- **Your trade-offs** — What did you skip, and what would you add with more time?
- **Scaling questions** — What happens under load? How would you handle concurrency?
- **Your AI usage** — How did you use AI tools? What did you adjust or rewrite?

We're not trying to catch you out. We want to understand how you think, communicate, and solve problems.

---

## Submission

Please share the following **at least 24 hours before the interview**:

- A link to a **Git repository** (GitHub, GitLab, etc.) with your code
  - A **README** with setup instructions
  - *(Optional)* A link to a deployed version
- 

## Technical Interview

You can book the meeting to discuss your coding challenge via the following link:

<https://cal.com/fgreinus/hubblr-technical-interview>

---

## Questions?

If anything is unclear, just ask — that's a good sign, not a bad one.

Good luck, and have fun with it! 🚀